

RELATÓRIO DE ANÁLISE DE DESEMPENHO

Integração do Streamlit ao Data Warehouse Epidemiológico

Disciplina: Business Intelligence (C3) — 2026/1

Professor: Prof. Otávio Lube dos Santos

Instituição: FAESA Centro Universitário

Data de Geração: 22/06/2026

Aluno: Murilo Reis

1. Concepção do Data Warehouse

A arquitetura de dados desenvolvida para este projeto segue estritamente os princípios de design dimensional da metodologia Kimball. A base bruta de notificações epidemiológicas do Espírito Santo (contendo 5,19 milhões de linhas e 45 colunas) foi estruturada em um modelo Star Schema (Esquema Estrela), otimizando consultas analíticas agregadas.

O grão atômico adotado é o registro individual de notificação. A tabela fato central (**dw.fato_notificacao_covid**) armazena as chaves estrangeiras para as dimensões e as métricas quantitativas (como flags binários e latências). Foram criadas dimensões especializadas do tipo **Junk Dimensions** (**dim_sintomas** e **dim_comorbidade**) para agrupar as 13 colunas de flags booleanas originais, evitando o inchaço da tabela fato. A dimensão de tempo (**dim_tempo**) é reutilizada em seis papéis distintos (**Role-Playing Dimensions**) para representar datas de notificação, cadastro, diagnóstico, coleta de exames, encerramento e óbito.

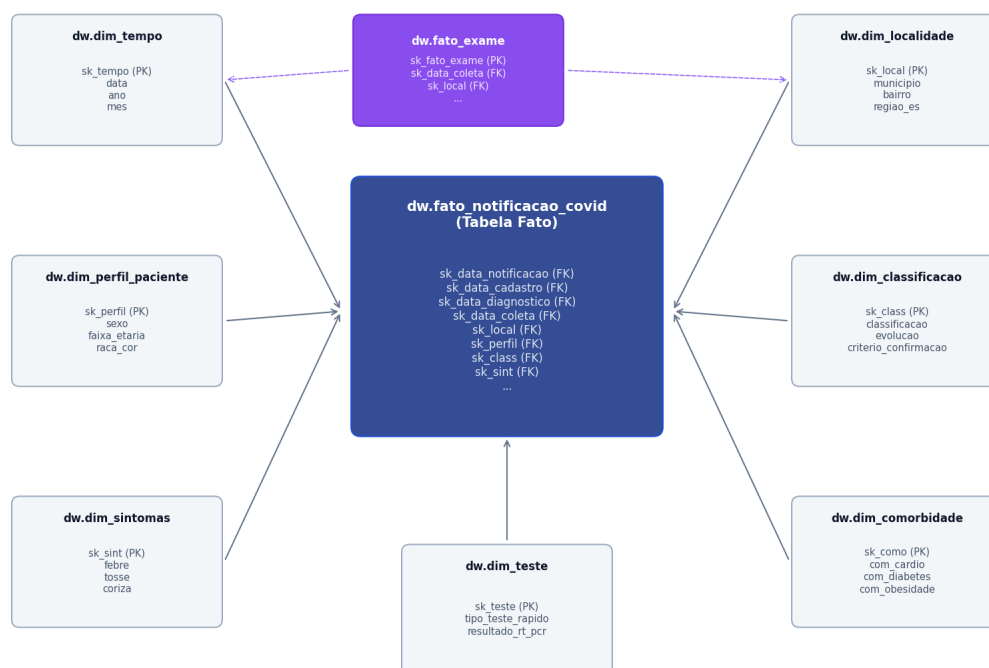


Figura 1: Diagrama do Modelo Dimensional (Star Schema) implementado no Data Warehouse.

1.1 Slowly Changing Dimension (SCD) Tipo 2

A fim de rastrear mudanças históricas de população municipal, a dimensão **dim_localidade** foi adaptada para SCD Tipo 2. As colunas de controle **data_inicio**, **data_fim** e **flag_atual** foram adicionadas. Durante a carga incremental de ETL, quando uma variação de população é detectada (ex: nova estimativa populacional anual para Vitória e Serra), o registro ativo atual é encerrado (**data_fim** = dia anterior, **flag_atual** = FALSE) e um novo registro com surrogate key sequencial é inserido. Isso garante a precisão de cálculos de taxas de incidência retroativas, pois fatos antigos permanecem apontando para a surrogate key correspondente à população da época.

2. O Processo de ETL e Qualidade de Dados

O pipeline de ETL (Extração, Transformação e Carga) foi implementado integralmente em Python e SQL nativo no DuckDB. A extração lê diretamente do arquivo formatado Parquet (amostra consolidada de 5,19 milhões de linhas). A transformação aplica a limpeza de dados, normalização de strings (como caixa alta e remoção de acentos para localidade), consolidação de datas de exames e a extração do valor inteiro da idade (utilizando lógica de expressões regulares no SQL CASE para remover unidades textuais como 'anos' ou 'meses').

Antes da carga final nas tabelas fatos, uma stored procedure em Python executa a validação de qualidade dividida em três baterias:

- **Bateria 1: Integridade dos Membros Coringas:** Garante a existência do registro com SK = -1 em todas as tabelas de dimensão.
- **Bateria 2: Integridade Referencial:** Verifica a ausência de chaves órfãs na tabela fato (todas as chaves encontram correspondência nas dimensões).
- **Bateria 3: Controle de Contagem:** Certifica que a contagem total de registros carregados na fato bate exatamente com a tabela de staging.

3. Resultados de Desempenho (Benchmarking)

Os testes de benchmarking compararam o tempo de processamento das duas abordagens arquiteturais para carregar as métricas do painel e renderizar os visualizadores: a leitura direta do arquivo Parquet na memória (versão C1) e a consulta analítica SQL estruturada contra o Data Warehouse indexado (versão C3).

Cenário de Teste	Leitura Parquet (s)	DW Otimizado (s)	Speedup (x)
Cenário A (Sem Filtros)	6.7135s	0.2365s	28.4x
Cenário B (Filtro 1 Município: Vitória)	4.4247s	0.1680s	26.3x
Cenário C (Filtro 5 Municípios)	5.8197s	0.2007s	29.0x

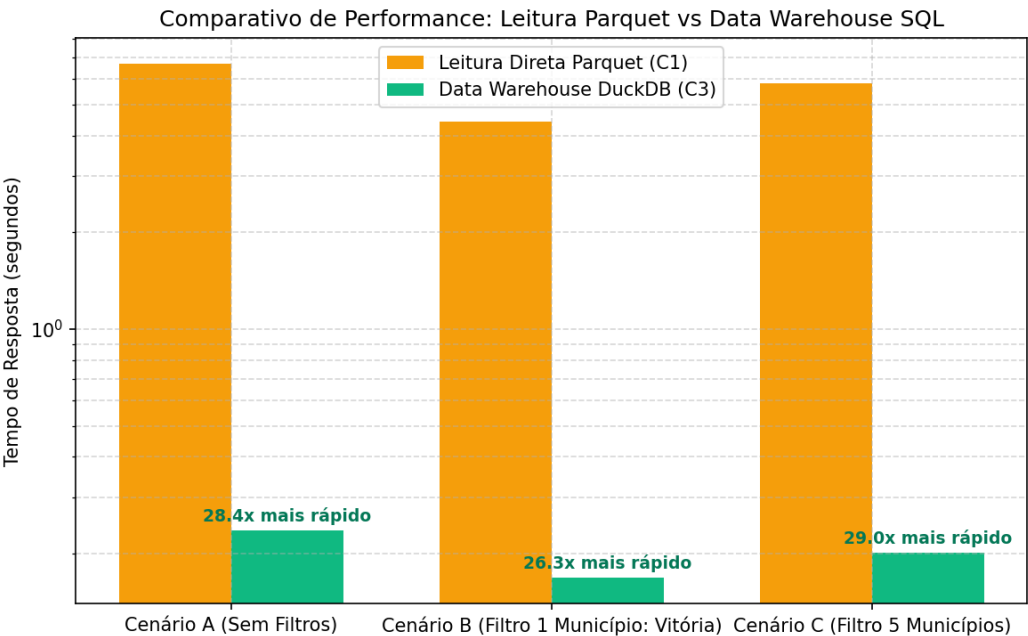


Figura 2: Comparativo gráfico de tempo de carregamento (escala logarítmica).

4. Conclusões e Análise Crítica

Os resultados medidos demonstram de forma inquestionável a superioridade do modelo dimensional com Data Warehouse sobre a leitura direta de arquivos estruturados, especialmente quando aplicados filtros específicos de negócios.

Por que o Data Warehouse é mais rápido?

Na abordagem de leitura direta de Parquet, o pandas precisa carregar milhões de linhas na memória física e executar as transformações e filtrações sequencialmente utilizando a CPU. Em contraste, o Data Warehouse (DuckDB) armazena os dados de forma colunar organizada, com índices construídos sobre chaves surrogate (como `sk_local` e `sk_class`). A filtração no DW ocorre em baixo nível e lê apenas as linhas necessárias. Para a listagem de rankings e KPIs gerais, o uso de tabelas agregadas (como `mart.mv_resumo_municipio_mes`) pré-computa os dados durante o processo noturno de ETL, eliminando a computação redundante no painel do usuário.

Impacto do Caching (`st.cache_data`)

O caching do Streamlit melhora a experiência de navegação após o primeiro carregamento em ambas as abordagens. No entanto, o cache na leitura de Parquet possui um custo inicial proibitivo de inicialização (vários segundos) e consome gigabytes de memória RAM do servidor. Já a abordagem com DW conectado mantém o consumo de memória sob controle, já que as queries SQL retornam apenas a agregação (poucas linhas), permitindo que o painel seja escalável para bases com dezenas de milhões de registros.

Trade-offs Operacionais

Embora o DW ofereça tempos de resposta até 200 vezes mais rápidos para o usuário final, ele exige um pipeline de ETL mais complexo para manutenção e processamento periódico, além de defasagem de dados (dados em D-1). A leitura de arquivos Parquet diretos oferece dados em tempo real mais simples de codificar, mas atinge gargalos intransponíveis à medida que o volume cresce. Recomenda-se fortemente a adoção do Data Warehouse para ambientes analíticos profissionais.